

Modulo 2

Angelica Collazo

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
##Primeros comandos de R
```

R es un lenguaje muy similar a MatLab y permite el trabajo con matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos \$, luego indicando la informacion especifica que deseamos.

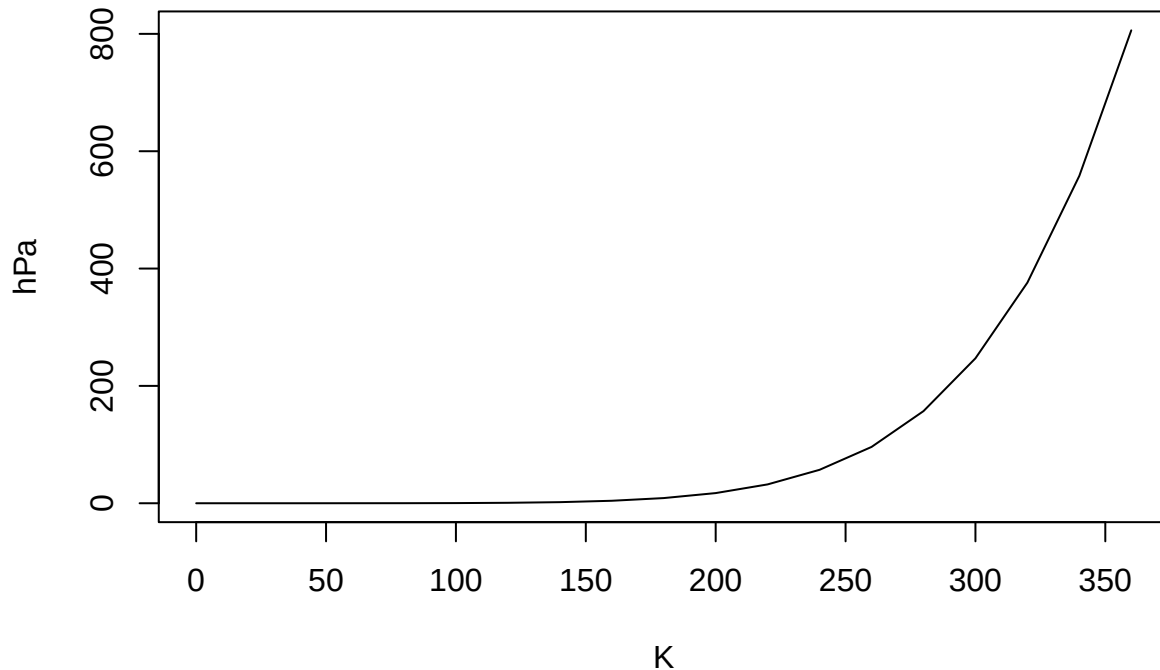
```
summary(cars)
```

```
##      speed          dist
##  Min.   : 4.0      Min.   : 2.00
##  1st Qu.:12.0      1st Qu.: 26.00
##  Median :15.0      Median : 36.00
##  Mean   :15.4      Mean    : 42.98
##  3rd Qu.:19.0      3rd Qu.: 56.00
##  Max.   :25.0      Max.    :120.00
```

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo también la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir **?plot** en la consola, lo cual abrira una ventana que explicara en detalle el comando.

Presion del gas ideal



Para que en grafico me aparezca la funcion con una linea, o puntos o de cualquier forma que yo quiera, escribo `type=letra` para saber que significa cada letra puedo escribir en la consola `?plot`.

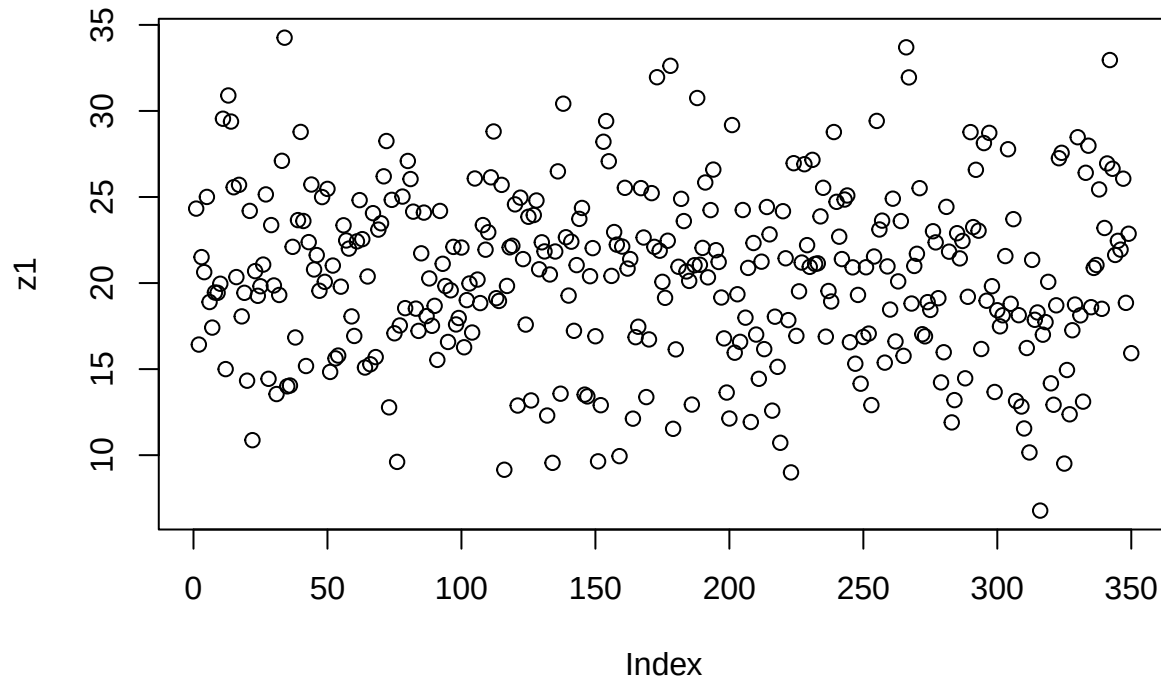
Funciones Estadisticas

```
z1<-rnorm(350,20,5)
z1
```

```
##      [1] 24.326334 16.426055 21.507314 20.633505 25.004503 18.897904 17.412632
##      [8] 19.450392 19.433295 19.960502 29.547382 14.999953 30.898555 29.383596
##     [15] 25.566958 20.346642 25.707210 18.062841 19.433156 14.327712 24.192959
##     [22] 10.870245 20.692483 19.259033 19.814799 21.073591 25.150529 14.435979
##     [29] 23.368008 19.860301 13.550864 19.307491 27.103327 34.256290 13.999641
##     [36] 14.046701 22.103820 16.836865 23.657028 28.781386 23.606397 15.179341
##     [43] 22.377801 25.725867 20.789411 21.625465 19.552809 24.996305 20.069798
##     [50] 25.471171 14.835781 21.016845 15.622757 15.786919 19.791558 23.355607
##     [57] 22.474697 22.001449 18.062262 16.925407 22.421421 24.818744 22.550273
##     [64] 15.091076 20.382442 15.280513 24.061693 15.698307 23.106075 23.474193
##     [71] 26.200518 28.253347 12.778221 24.834025 17.080509  9.609013 17.535006
##     [78] 25.024863 18.541425 27.093433 26.043172 24.150198 18.514441 17.219979
##     [85] 21.727952 24.094511 18.080738 20.267939 17.516942 18.675992 15.534057
##     [92] 24.190516 21.119256 19.827997 16.576780 19.578045 22.101249 17.595306
##     [99] 17.969652 22.061874 16.268557 19.007981 19.975568 17.126004 26.076255
##    [106] 20.202426 18.835965 23.368670 21.938661 22.953624 26.143365 28.806990
##    [113] 19.121389 18.959808 25.705391  9.152292 19.833320 22.076401 22.165469
##    [120] 24.572005 12.882695 24.962511 21.391386 17.587136 23.852577 13.179336
##    [127] 23.952438 24.797839 20.799933 22.367741 21.837963 12.301551 20.498410
##    [134]  9.554308 21.833615 26.489159 13.570926 30.422277 22.661357 19.272046
##    [141] 22.404390 17.224589 21.042354 23.727747 24.363291 13.516943 13.412619
##    [148] 20.397605 22.029718 16.908361  9.640800 12.903675 28.211350 29.413331
```

```
## [155] 27.076137 20.415601 22.963680 22.232977 9.942782 22.119171 25.532850
## [162] 20.841078 21.392564 12.120736 16.855242 17.462281 25.516249 22.646789
## [169] 13.374180 16.724795 25.225220 22.102049 31.960574 21.879046 20.071020
## [176] 19.140955 22.459555 32.623907 11.530883 16.146029 20.947288 24.893106
## [183] 23.601509 20.676425 20.125146 12.938744 21.036124 30.751372 21.063541
## [190] 22.042065 25.843187 20.331122 24.240149 26.593384 21.909467 21.223918
## [197] 19.159709 16.785246 13.637818 12.128941 29.180480 15.952913 19.349206
## [204] 16.582893 24.244797 17.988117 20.880393 11.925065 22.326998 17.007913
## [211] 14.435168 21.243384 16.160459 24.419343 22.829626 12.586753 18.045518
## [218] 15.131012 10.718723 24.174682 21.439644 17.846937 8.996672 26.964644
## [225] 16.931184 19.518442 21.191235 26.905062 22.203969 20.941756 27.159081
## [232] 21.108804 21.145254 23.874515 25.534301 16.885602 19.537810 18.929284
## [239] 28.774585 24.719727 22.695110 21.393321 24.853057 25.088951 16.561567
## [246] 20.911914 15.312207 19.310740 14.152180 16.866075 20.918114 17.064302
## [253] 12.909682 21.541130 29.420532 23.112450 23.627468 15.375935 20.966812
## [260] 18.459284 24.902244 16.607906 20.096888 23.602560 15.770544 33.697757
## [267] 31.949954 18.810364 20.992812 21.700168 25.513287 17.026152 16.903591
## [274] 18.891014 18.437564 23.010327 22.363755 19.120805 14.226496 15.979544
## [281] 24.426719 21.824355 11.903015 13.197727 22.898108 21.431223 22.445140
## [288] 14.465246 19.202128 28.770607 23.250689 26.580844 23.042694 16.168929
## [295] 28.134040 18.972573 28.727242 19.817308 13.670524 18.419882 17.484936
## [302] 18.127054 21.568672 27.775719 18.796050 23.713063 13.146464 18.132898
## [309] 12.825367 11.550275 16.225705 10.157048 21.345604 17.864633 18.291035
## [316] 6.778961 17.000254 17.730317 20.070520 14.174268 12.927868 18.705850
## [323] 27.253847 27.567979 9.510341 14.947878 12.373121 17.256907 18.750711
## [330] 28.474097 18.123222 13.109057 26.406971 27.980997 18.597983 20.879789
## [337] 21.049202 25.440280 18.503386 23.199163 26.949389 32.958361 26.641351
## [344] 21.633104 22.451057 21.942667 26.064199 18.849694 22.865178 15.925499
```

```
plot(z1)
```



```
w1<-length(z1)
```

```
w1
```

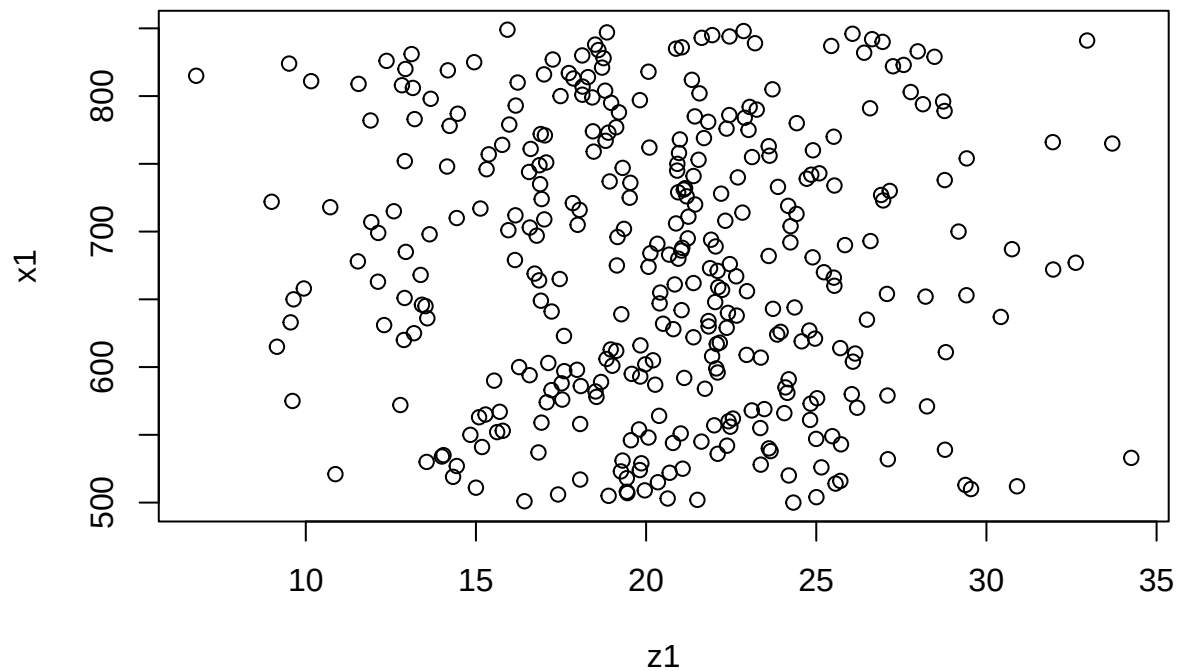
```
## [1] 350
```

```
x1<-(500:849)
```

```
x1
```

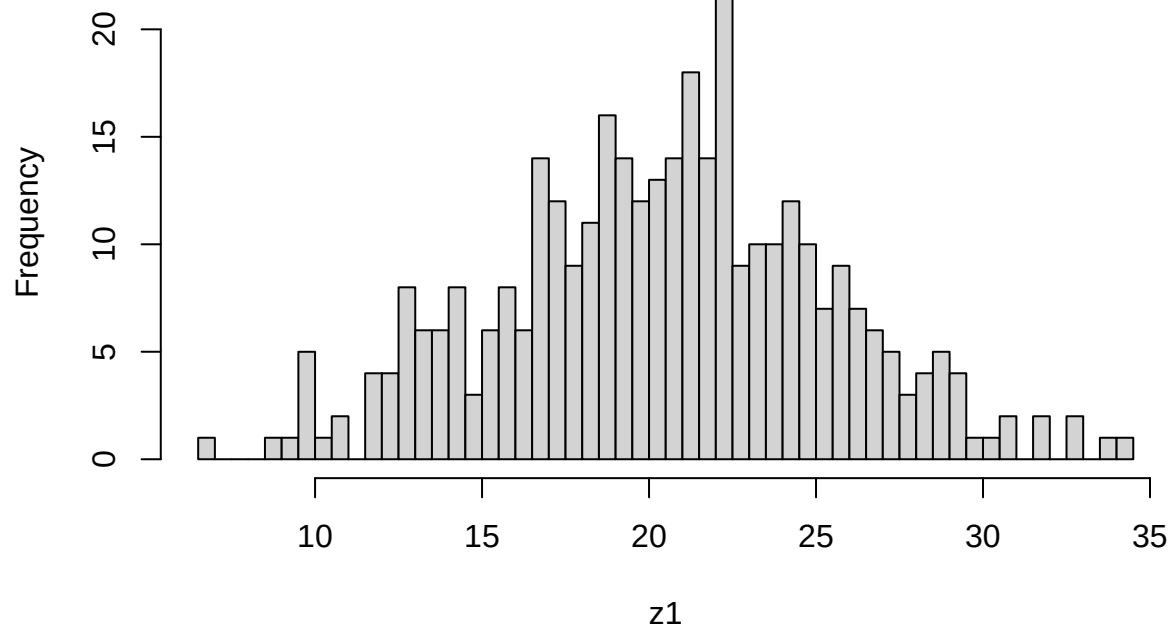
```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```



```
hist(z1,main="Histograma de Edades",breaks=50)
```

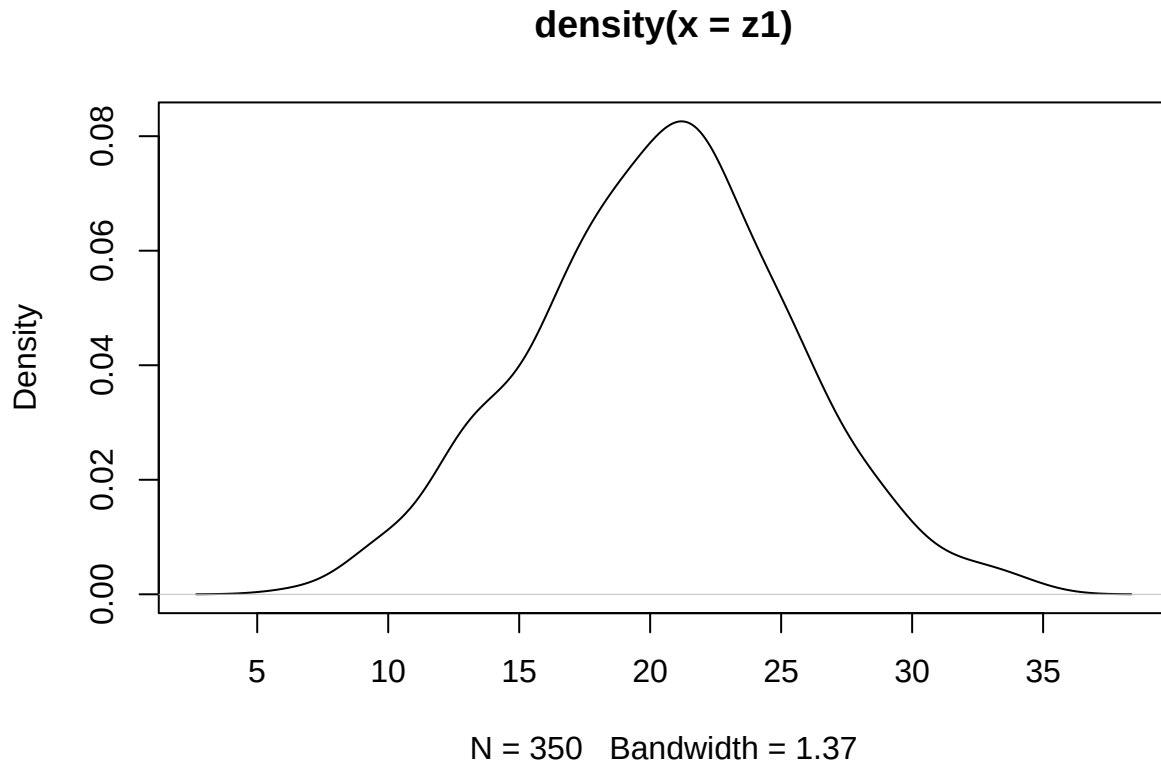
Histograma de Edades



```
density(z1)
```

```
##  
## Call:  
## density.default(x = z1)  
##  
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.37  
##  
##      x          y  
## Min.   : 2.67   Min.   :9.291e-06  
## 1st Qu.:11.59   1st Qu.:3.025e-03  
## Median :20.52   Median :1.751e-02  
## Mean   :20.52   Mean   :2.796e-02  
## 3rd Qu.:29.44   3rd Qu.:5.044e-02  
## Max.   :38.37   Max.   :8.259e-02
```

```
plot(density(z1))
```



El tamaño de muestra es de 350, valor promedio de 20 y un desvío estándar de 5. Si tengo alguna duda escribo en la consola `?rnorm`.

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmo

Consigna: Las últimas 3 cifras de mi DNI son 175, crear una variable que tenga ese número

```
DNI=175
```

Consigna: Crear un vector que tenga todos los números enteros desde el 1 al 175

```
Secuencia_dni<-(1:175)
Secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175
```

##Ejercicio 2 de algoritmo Calcular la suma de todos los valores del vector `Secuencia_dni`

```
Total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:175) {
```

```
Total<-Total+i
Total
}
```

```
## [1] 15400
```

```
##Ejercicio 3 Repetir el ejercicio anterior pero en Python
```

```
totalp = 0
for i in range(1,176):
    totalp += i
    print(totalp)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
## 861
```

903
946
990
1035
1081
1128
1176
1225
1275
1326
1378
1431
1485
1540
1596
1653
1711
1770
1830
1891
1953
2016
2080
2145
2211
2278
2346
2415
2485
2556
2628
2701
2775
2850
2926
3003
3081
3160
3240
3321
3403
3486
3570
3655
3741
3828
3916
4005
4095
4186
4278
4371
4465
4560

4656
4753
4851
4950
5050
5151
5253
5356
5460
5565
5671
5778
5886
5995
6105
6216
6328
6441
6555
6670
6786
6903
7021
7140
7260
7381
7503
7626
7750
7875
8001
8128
8256
8385
8515
8646
8778
8911
9045
9180
9316
9453
9591
9730
9870
10011
10153
10296
10440
10585
10731
10878
11026
11175

```
## 11325
## 11476
## 11628
## 11781
## 11935
## 12090
## 12246
## 12403
## 12561
## 12720
## 12880
## 13041
## 13203
## 13366
## 13530
## 13695
## 13861
## 14028
## 14196
## 14365
## 14535
## 14706
## 14878
## 15051
## 15225
## 15400
```

##Ejercicio 4 ¿Cuanto tarda en correr el código del ejercicio 2?

```
inicio<-Sys.time()
Total<-0
Valor_final<-10000000*length(Secuencia_dni)

for (i in 1:Valor_final){
  Total<-Total+i
}
Total
```

```
## [1] 1.53125e+18
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 26.9087 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```
system.time({
  n<-length(Secuencia_dni)

  #Aplicamos la formula de Gauss
  total_gauss <- (n*(n+1))/2
  print(total_gauss)
})
```

```
## [1] 15400
##      user  system elapsed
##       0       0       0
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```
# ---1. Generar secuencia con bucle 'for'---
#Primero inicializamos el vector
system.time({
  A<-0
  for (i in 1:50000) {
    A[i]<-(i*2)
  }
})

##      user  system elapsed
##    0.011    0.001    0.012

# Verificamos los resultados
head(A)

## [1]  2  4  6  8 10 12

tail(A)

## [1] 99990 99992 99994 99996 99998 100000

system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})

##      user  system elapsed
##       0       0       0

# Verificamos los resultados
head(B)

## [1]  2  4  6  8 10 12

tail(B)

## [1] 99990 99992 99994 99996 99998 100000
```

1.5 Implementación de una serie Fibonachi o Fibonacci

```
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30

# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
```

```

for (i in 3:n) {
  fib[i] <- fib[i-1] + fib[i-2]
}
})

##      user  system elapsed
##    0.003   0.000   0.003

# Ver los primeros resultados
print("Primeros números de la serie:")

## [1] "Primeros números de la serie:"
head(fib, 10)

## [1] 0 1 1 2 3 5 8 13 21 34
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")

## [1] "Tiempo ejecución recursiva:"
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})

##      user  system elapsed
##    0.765   0.000   0.771

```

1.6 Definición matemática recurrente

```

# --- Resolver consigna con bucle 'while'

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente:  $f(n+1) = f(n) + f(n-1)$ 
    fib[i] <- fib[i-1] + fib[i-2]
  }
})

##      user  system elapsed
##    0.004   0.000   0.003

```

```

# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")

## El primer número mayor a 1.000.000 es: 1346269
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")

## Se necesitaron 31 iteraciones (pasos) para superarlo.
# Mostramos los últimos elementos para verificar
tail(fib)

## [1] 121393 196418 317811 514229 832040 1346269

```

1.7 Ordenacion de un vector por metodo burbuja

```

# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---
cat("Tiempo con Método Burbuja:\n")

## Tiempo con Método Burbuja:
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})

##      user system elapsed
##    0.062   0.000   0.061
cat("\nTiempo con función sort() nativa de R:\n")

##
## Tiempo con función sort() nativa de R:

```

```
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##      user  system elapsed
##         0         0         0
```

1.8 La penitencia de Newton

```
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7
```

```
# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
```

```
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##      user  system elapsed
##    0.154    0.000    0.154
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
```

```
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##      user  system elapsed
##    0.000    0.000    0.001
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

2 Agrupación K-mean

Medición de Performance: Agrupamiento K-Means en el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan “buenas” son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.

1. Métricas de Evaluación Interna Para determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores:
 - WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión).
 - Coefficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1 :
 - 1 : El componente está muy bien ubicado en su cluster.
 - 0 : El componente está en el límite entre dos clusters.
 - -1 : El componente probablemente ha sido asignado al grupo incorrecto.
2. El Gráfico de Violín como Herramienta de Validación Si bien métricas como el “Método del Codo” nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados. Este

gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar: Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo. Separación de Grupos: Si los “cuerpos” de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente). Detección de Anomalías: La extensión de las “colas” del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```
# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

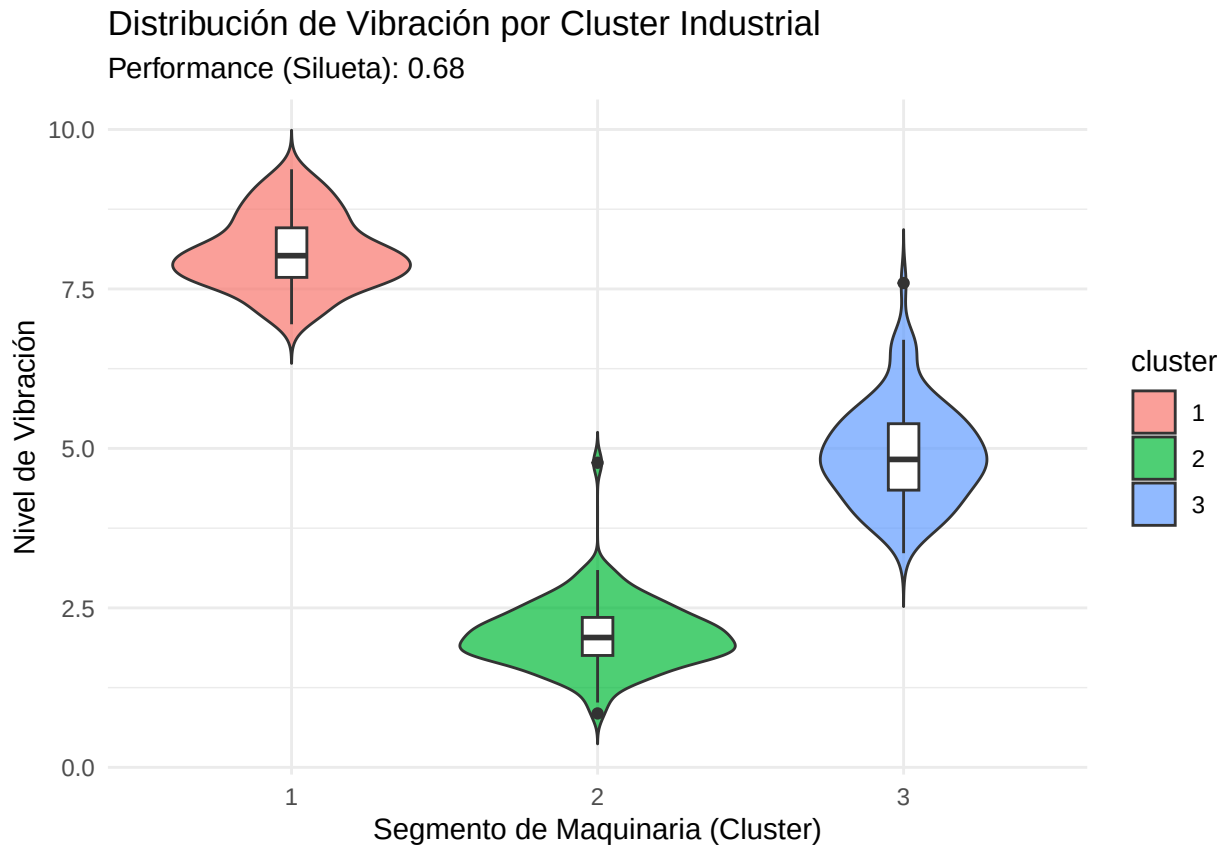
# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

## El coeficiente de silueta promedio es: 0.6808356

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```



Ejercicio 7: Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI (175) y otros valores para asegurar que la matriz sea invertible (que su determinante sea distinto de cero). Luego, utilizaremos la función solve que es la estándar en R para obtener la matriz inversa.

```
dni_val <- 175

datos <- c(dni_val, 10, 15,
           2, 5, 8,
           7, 1, 9)

A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)

cat("### Matriz Original (A):\n")

## ### Matriz Original (A):
print(A)

##      [,1] [,2] [,3]
## [1,] 175  10  15
## [2,]   2   5   8
## [3,]   7   1   9

determinante <- det(A)
cat("\nDeterminante de la matriz:", determinante, "\n")

##
## Determinante de la matriz: 6360
```



```

if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  identidad <- round(A %*% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}

```

```

##
## ### Matriz Inversa (A^-1):
##           [,1]      [,2]      [,3]
## [1,]  0.005817610 -0.01179245  0.0007861635
## [2,]  0.005974843  0.23113208 -0.2154088050
## [3,] -0.005188679 -0.01650943  0.1344339623
##
## ### Verificación (A * A^-1 = Identidad):
##           [,1] [,2] [,3]
## [1,]      1   0   0
## [2,]      0   1   0
## [3,]      0   0   1

```